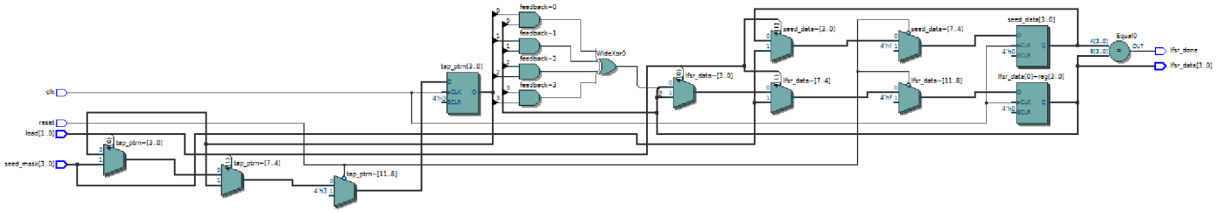
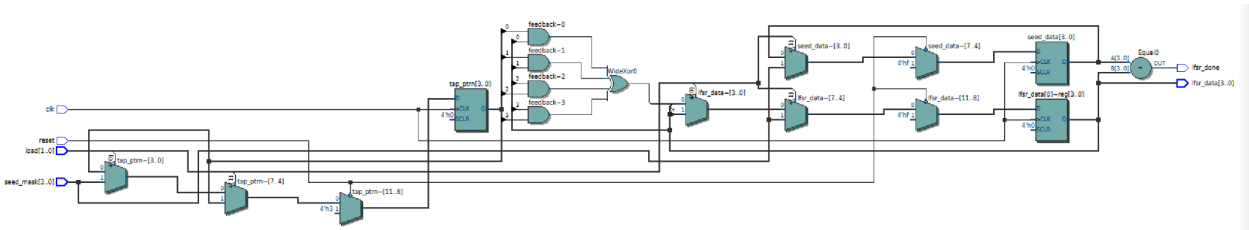
Synthesis:
(Unclear which to do so here, I have for both N=4 and 6) [I made this before the clarification]

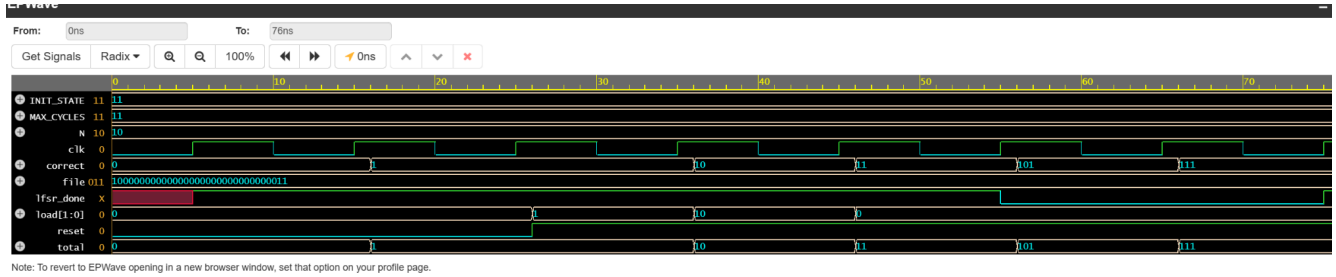


	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	9
2		
3	Combinational ALUT usage for logic	16
1	-- 7 input functions	0
2	-- 6 input functions	1
3	-- 5 input functions	2
4	-- 4 input functions	1
5	-- <=3 input functions	12
4		
5	Dedicated logic registers	12
6		
7	I/O pins	13
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	res...put
12	Maximum fan-out	13
13	Total fan-out	107
14	Average fan-out	1.98

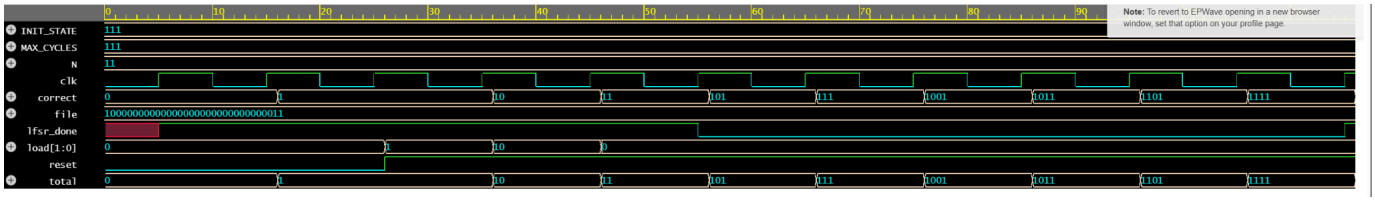


	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	9
2		
3	▼ Combinational ALUT usage for logic	16
1	-- 7 input functions	0
2	-- 6 input functions	1
3	-- 5 input functions	2
4	-- 4 input functions	1
5	-- <=3 input functions	12
4		
5	Dedicated logic registers	12
6		
7	I/O pins	13
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	res...put
12	Maximum fan-out	13
13	Total fan-out	107
14	Average fan-out	1.98

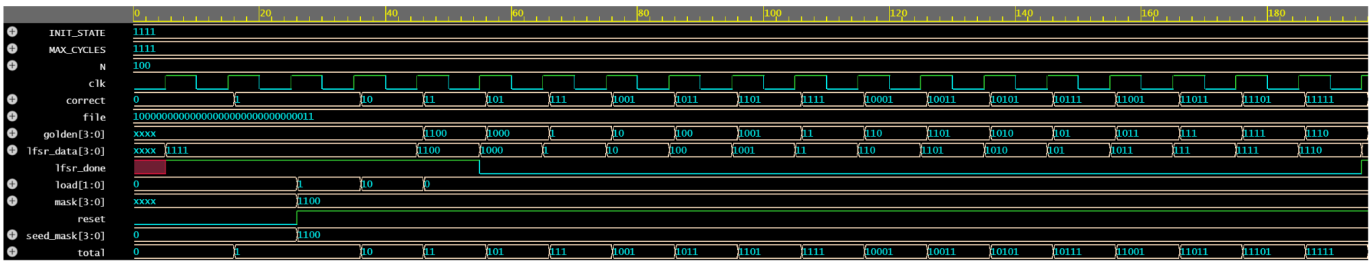
As I would guess, the synthesis didn't change because the module was designed to be … modular and handle all required inputs and work the same.



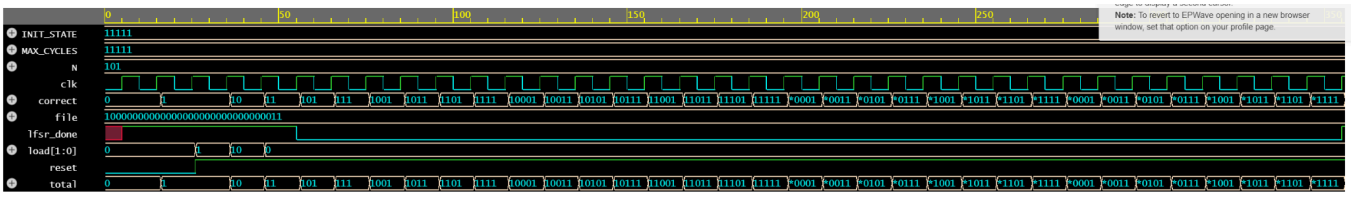

```
# Testbench done: passed 9 of 9 checks
```



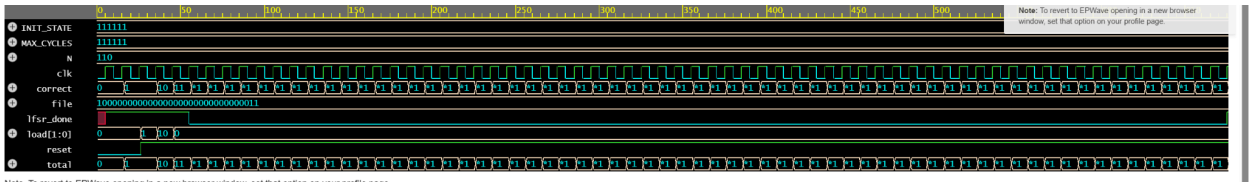

```
# Testbench done: passed 17 of 17 checks
```



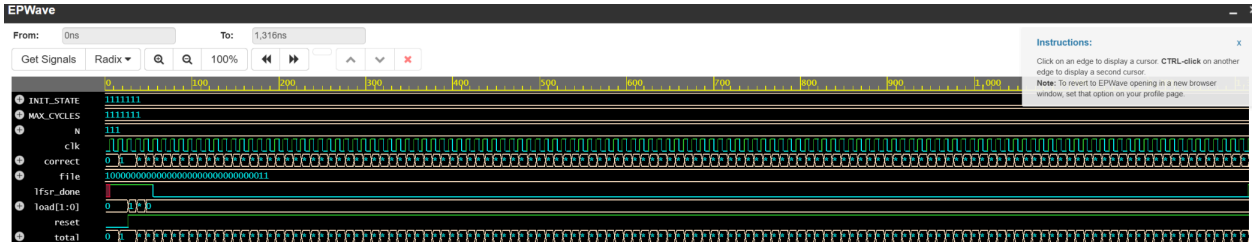

```
# Testbench done: passed 33 of 33 checks
# ** Note: $finish    : testbench.sv(136)
#    Time: 196 ns  Iteration: 0  Instance: /lfsr_testbench3
# End time: 21:51:04 on Feb 15,2026, Elapsed time: 0:00:01
# Errors: 0, Warnings: 0
End time: 21:51:04 on Feb 15,2026, Elapsed time: 0:00:02
```



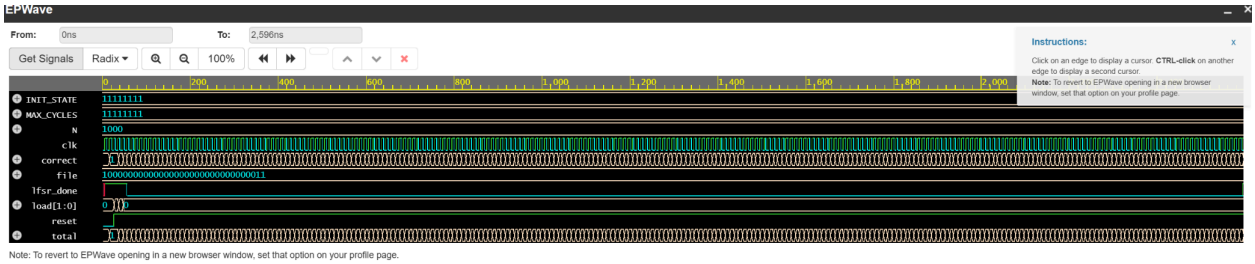

```
# Testbench done: passed 65 of 65 checks
```




```
# Testbench done: passed 129 of 129 checks
```




```
# Testbench done: passed 257 of 257 checks
```



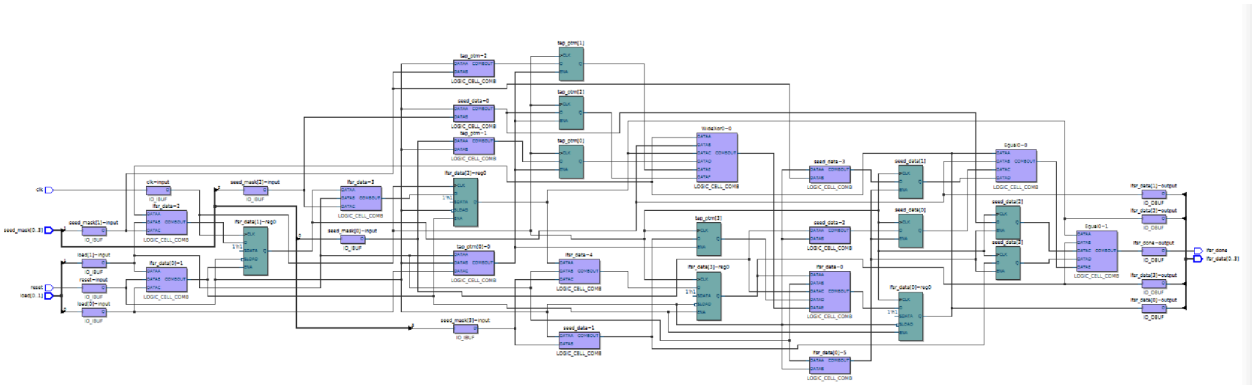

```
# Testbench done: passed 513 of 513 checks
```

Above is the simulation (and verification) of all N 2-8 (because instructions were unclear, so I went with the more inclusive option).

The testbench loops through all iterations and checks each bitstring, and by the output, we can tell that the LFSR works as it is intended. The message to console makes it easy to see that it passed the autograder tests.

The module works as follows. On reset low active signal, `lfsr_data` and `seed_data` are set to all 1s. `tap_ptrn` is set to the default tap mask for the chosen N).
If Load seed (load[1] = 1, `lfsr_data` and `seed_data` are set to `seed_mask`. This is the value that `lfsr_done` will compare against when the sequence returns. When Load tap pattern (load[0] = 1) `tap_ptrn` is set to `seed_mask`.A 1 in bit i of `tap_ptrn` means bit i of `lfsr_data` is used in the XOR feedback.
On a Normal step (no load), feedback bit = XOR of all bits where `tap_ptrn` is 1: `feedback = ^(lfsr_data & tap_ptrn)`. Register shifts left, and the vacated LSB is filled with `feedback`: `lfsr_data <= {lfsr_data[N-2:0], feedback}`.

For a visual logic representation here is the post-mapping schematic:



Part 2

For part 2,

N	Bitstrings (Hex/Binary Tap Masks)	Total Count
2	2'b11	1
3	3'b110, 3'b101	2
4	4'b1100, 4'b1001	2
5	5'b10100 , 5'b10010 , 5'b11110 , 5'b11101 , 5'b11011, 5'b10111	6
6	6'b110000 , 6'b100001 , 6'b110011 , 6'b111101 , 6'b110110 , 6'b101101	6
7	7'b1100000 7'b1000001 , 7'b1110000 , 7'b1000111 , 7'b1100100 , 7'b1001001 , 7'b1011111 , 7'b1111101 , 7'b1101100, 7'b1001101 , 7'b1111011 , 7'b1101111 , 7'b1011000 , 7'b1000101, 7'b1110100, 7'b1010111 , 7'b1111110 , 7'b1111111	18
8	8'b10111000, 8'b10001110, 8'b10110100 , 8'b11010100 , 8'b10010110, 8'b10100111, 8'b11100011 , 8'b10111101 , 8'b11011101 , 8'b11101011 , 8'b11101101 , 8'b11011011 , 8'b10111111 , 8'b11111101 , 8'b11000111 , 8'b11100111	16